

Route Optimizer — Technical Analysis & Change Log

Route Optimizer — Technical Analysis & Change Log

FirstKnock Door-Knocking App | routeOptimizer.ts

April 8, 2026

Figure 1 — FirstKnock Route 1 (84 stops, Dallas) showing visible yellow line crossings caused by the two compounding bugs described in Section 1

2Root Cause Bugs Fixed	7Functions Added or Changed	11timesFewer Distance Calculations (100-prop session)	1Line Changed in generateOptimizedRoutes
------------------------	-----------------------------	---	--

The visible crossing yellow lines on the map are caused by two compounding bugs: (1) inter-street ordering using nearest individual property instead of nearest street centroid, and (2) global 2-Opt destroying street groupings after they were established. The fix is a complete replacement of the ordering pipeline with a mail-carrier (boustrophedon) algorithm, triggered by a single line change in generateOptimizedRoutes.

Section 1 — Root Cause Analysis

The Backtracking Problem

The screenshot shows yellow route lines visibly crossing back over themselves on the map. This is a classic symptom of two compounding bugs in the original implementation. Each bug is independently capable of producing backtracking; together, they reinforce each other in a way that makes the final route neither properly street-grouped nor globally optimal.

Bug 1: orderForStreetSweep Used Property-Level Nearest-Neighbor for Inter-Street Ordering

The original code called orderForStreetSweep(clusterProps) from territoryLogic.ts. While this function correctly groups properties by street name, its inter-street ordering logic selects the next street by finding the nearest individual property across all remaining streets — not the nearest street centroid.

The consequence: the algorithm might be on the far end of Oak Street, and instead of crossing to the nearest street (Maple, whose centroid is 200 feet away), it jumps to Elm Street because one house on Elm happens to be 180 feet from the current position — even though the rest of Elm Street is half a mile away. The route then has to backtrack across Oak and Maple to finish Elm.

This is the primary source of the crossing yellow lines visible in the screenshot.

Bug 2: Global 2-Opt and Link Swap Destroyed Street Groupings

After `orderForStreetSweep` produced its output, the original `generateOptimizedRoutes` called `apply2Opt` and `applyLinkSwap` on the entire route — all properties across all streets simultaneously. These algorithms optimize purely for total distance, with no awareness of street boundaries. A 2-Opt swap that saves 50 feet of total distance might interleave properties from three different streets, completely destroying the street-by-street grouping that `orderForStreetSweep` had just established.

The result: even when the street ordering was correct, the global optimization pass re-introduced the exact backtracking it was supposed to prevent.

Why These Two Bugs Compound

The two bugs reinforce each other in a destructive feedback loop. The poor inter-street ordering from Bug 1 creates a route with many long cross-street jumps. The global 2-Opt from Bug 2 then attempts to fix those jumps by swapping individual properties — which breaks street groupings further. The final route is neither properly street-grouped nor globally optimal. It is the worst of both worlds, producing the visible crossing pattern on the map.

Table 1 — Root Cause Bug Summary

Bug	Location	Mechanism	Symptom
Bug 1	<code>orderForStreetSweep</code> in <code>territoryLogic.ts</code>	Nearest individual property used for inter-street selection instead of nearest street centroid	Route jumps to distant streets because one house is close, forcing backtrack to finish skipped streets
Bug 2	<code>generateOptimizedRoutes</code> in <code>routeOptimizer.ts</code>	Global <code>apply2Opt</code> and <code>applyLinkSwap</code> called on entire route, ignoring street boundaries	Street groupings destroyed post-ordering; properties from multiple
Compound Effect	Both files	Bug 1 creates long cross-street jumps; Bug 2 attempts to fix them by breaking groupings further	Final route is neither street-grouped nor globally optimal — visible crossing yellow lines on map

Section 2 — What Changed and Why

The following table summarizes every function that was added, changed, or removed in this

rewrite. Detailed before/after analysis follows for each function.

Table 2 — Function Change Summary

Function	Status	File	Purpose
normalizeStreetName	NEW	routeOptimizer.ts	Strips directional prefixes and type suffixes to unify street name variants
groupByStreet	NEW	routeOptimizer.ts	Builds Map<string, Property[]> keyed by normalized street name
orderStreetsByNearestNeighbor	NEW	routeOptimizer.ts	Centroid-based nearest-neighbor TSP with 2-Opt on centroid sequence
boustrophedonStreet	NEW	routeOptimizer.ts	Mail-carrier odd/even side split with directional U-shape walk
applyIntraStreet2Opt	NEW	routeOptimizer.ts	2-Opt scoped strictly to a single street's properties
mailCarrierOrder	NEW EXPORT	routeOptimizer.ts	Orchestrates full pipeline: group rightarrow order streets rightarrow boustrophedon
generateOptimizedRoutes	CHANGED (1 line)	routeOptimizer.ts	Swaps orderForStreetSweep for mailCarrierOrder; removes global apply2Opt and applyLinkSwap
optimizeRouteByDistance	UNCHANGED	routeOptimizer.ts	Standalone distance-only utility; global 2-Opt appropriate here
orderForStreetSweep	REMOVED (import)	territoryLogic.ts	No longer imported or called from routeOptimizer.ts

normalizeStreetName (NEW)

Before	Did not exist. Street grouping used raw street_name strings, so 'Oak St', 'Oak Street', 'N Oak St', and 'N Oak Street' were treated as four different streets, fragmenting what should be a single walking pass into four tiny groups.
After	Strips leading directional prefixes (N, S, E, W, NE, NW, SE, SW) and trailing type suffixes (ST, AVE, BLVD, DR, RD, LN, WAY, CT, PL, CIR, STREET, AVENUE, BOULEVARD, DRIVE, ROAD, LANE, COURT, PLACE, CIRCLE). 'N Oak St' and 'Oak Street' both normalize to 'OAK', ensuring all houses on the same physical street are grouped together.
Why it fixes backtracking	Fragmented street groups mean the algorithm treats one physical street as multiple micro-streets, forcing unnecessary cross-street jumps between what are actually adjacent houses.

groupByStreet (NEW)

Before	Did not exist as a standalone function. Street grouping was embedded inside orderForStreetSweep in territoryLogic.ts.
After	A clean, pure function that builds a Map<string, Property[]> keyed by normalized street name. Properties with missing or empty street_name are placed in a special __UNKNOWN__ group that is always appended last.

Why it fixes backtracking	Centralizes grouping logic, making it testable and ensuring consistent behavior. The <code>__UNKNOWN__</code> group handling prevents properties with bad data from disrupting the ordered street sequence.
----------------------------------	---

orderStreetsByNearestNeighbor (NEW — replaces orderForStreetSweep inter-street logic)

Before	<code>orderForStreetSweep</code> selected the next street by finding the nearest individual property across all remaining unvisited properties. This caused the algorithm to jump to a distant street because one of its houses happened to be close, ignoring that the rest of that street was far away.
---------------	---

After	Computes the geographic centroid (mean lat/lng) of each street group, then runs nearest-neighbor TSP on the centroids — not on individual properties. After nearest-neighbor produces an initial ordering, 2-Opt is applied to the centroid sequence to tighten the inter-street tour. This is safe because 2-Opt on centroids only reorders streets; it never mixes properties across street boundaries.
--------------	---

Why it fixes backtracking	By routing to the nearest street (centroid) rather than the nearest house, the algorithm commits to completing an entire street before moving on. The 2-Opt pass on centroids further reduces the total distance traveled between streets without any risk of breaking street groupings.
----------------------------------	--

boustrophedonStreet (NEW)

Before	Did not exist. Within a street, properties were ordered by a generic nearest-neighbor pass that ignored which side of the street each house was on, causing the rep to zigzag back and forth across the street.
After	Implements the true mail-carrier pattern. Determines the street's primary axis (E-W if bearing is 45–135° or 225–315°, otherwise N-S). Splits properties into odd-numbered and even-numbered sides. Sorts each side by axis coordinate. Determines which end of the street the rep is arriving from (by comparing distance to the low-end vs high-end house). Walks the first side in the arrival direction, then crosses and walks the other side in reverse — completing a full U-shape with no backtracking.
Why it fixes backtracking	This is the core fix for intra-street crossing lines. Instead of zigzagging across the street, the rep walks a clean boustrophedon (serpentine) pattern: all of one side, then all of the other.

applyIntraStreet2Opt (NEW — replaces global apply2Opt)

Before	apply2Opt was called on the entire route (all properties across all streets). This global optimization destroyed street groupings to save small amounts of total distance.
After	applyIntraStreet2Opt receives only the properties for a single street and applies 2-Opt strictly within that set. Because it only ever sees one street's properties, cross-street swaps are structurally impossible. The 300-node safety limit is preserved.

Why it fixes backtracking	Intra-street 2-Opt tightens the walking path within each street (e.g., handling cul-de-sacs or irregular lot layouts) without any risk of interleaving properties from different streets.
----------------------------------	---

mailCarrierOrder (NEW EXPORT)

Before	Did not exist. No single function encapsulated the complete mail-carrier ordering pipeline.
---------------	---

After	Exported function that orchestrates the full pipeline: <pre>groupByStreet rightarrow orderStreetsByNearestNeighbor rightarrow boustrophedonStreet rightarrow applyIntraStreet2Opt rightarrow concatenate.</pre> Tracks the exit point (last property) of each street and passes it as the arrival point for the next street, ensuring the boustrophedon direction is always correct relative to where the rep is coming from.
--------------	---

Why it fixes backtracking	Provides a clean, testable entry point for the entire algorithm. The exit-point tracking is critical — without it, boustrophedonStreet would always assume arrival from the same direction, potentially causing unnecessary U-turns at street transitions.
----------------------------------	--

generateOptimizedRoutes (CHANGED — one line)

Before	<pre>let orderedProps = orderForStreetSweep(clusterProps); followed by global apply2Opt and applyLinkSwap calls.</pre>
---------------	--

After	let orderedProps = mailCarrierOrder(clusterProps, startLocation); followed by fatigueAwareFrontLoad(orderedProps). The global apply2Opt and applyLinkSwap calls are removed. The import of orderForStreetSweep is removed from the territoryLogic import line.
Why	The single line change swaps the broken ordering algorithm for the correct one. Removing the global optimization passes prevents them from re-introducing backtracking after the mail-carrier algorithm has produced a clean route.

optimizeRouteByDistance (UNCHANGED)

This function is a standalone utility used for distance-only route optimization (not the main door-knocking flow). It retains apply2Opt and applyLinkSwap because it operates on a flat property list without street-grouping requirements — global optimization is appropriate and correct in this context. No changes were made.

Section 3 — The Mail Carrier Algorithm Explained

This section is written in plain language for non-engineering stakeholders. No prior knowledge of routing algorithms is assumed.

The Old Way: Jumping Around Like a Pinball

Imagine you need to mow a large lawn that has many separate rectangular sections. The old algorithm would look at where you are standing right now, find the single closest patch of uncut grass anywhere on the entire lawn, mow that patch, then look around again for the next closest patch. This sounds logical, but it means you end up running back and forth across the lawn constantly — mowing a patch on the left side, then one on the right, then back to the left, leaving a chaotic trail of footprints that cross over each other dozens of times.

That is exactly what the yellow crossing lines on the map represent. The sales rep is being sent to the nearest individual house at each step, regardless of whether that house is on a completely different street that will require backtracking to finish.

The New Way: Mowing in Rows

A professional groundskeeper mows in rows. They start at one end of a section, mow the entire length, turn around, mow back the other way, and keep going until the whole section is done. Then

they walk to the nearest unfinished section and repeat. No backtracking, no crossing paths. The new algorithm works the same way, in four steps.

1. Find all the 'sections' (streets). Every property is grouped by its street name. 'Oak St', 'N Oak Street', and 'Oak Avenue' all get recognized as the same street — 'OAK' — so the rep doesn't get sent back to the same block multiple times under different names.
2. Plan the most efficient order to visit each street. Instead of jumping to whichever individual house is closest, the algorithm finds the geographic center of each street and plans a route between those centers. This is like planning which lawn sections to mow in what order — you finish one whole section before moving to the next nearest one.
3. Walk each street like a mail carrier. On each street, the rep walks all the houses on one side (say, the odd-numbered side: 101, 103, 105, 107), then crosses the street and walks back the other way on the even side (108, 106, 104, 102). This is the boustrophedon pattern — the same way ancient Greeks wrote text, alternating direction on each line. No zigzagging across the street.
4. Fine-tune within each street. A small optimization pass tidies up any irregularities within a single street (like a cul-de-sac at the end) without ever mixing up houses from different streets.

The result: the rep walks in clean, parallel passes with no backtracking. The yellow lines on the map will no longer cross.

Table 3 — Old vs. New Algorithm Behavior by Step

Step	What It Does	Old Behavior	New Behavior
1 — Group streets	Identify which properties belong to the same street	Raw string match — 'Oak St' and 'Oak Street' treated as different streets	Normalized match — all variants of the same street unified into one group
2 — Order streets	Decide which street to visit next	Jump to street containing the nearest individual house	Jump to street whose geographic center is nearest
3 — Walk each street	Determine order within a street	Nearest-neighbor zigzag across both sides of the street	Mail-carrier U-shape: all of one side, then all of the other
4 — Fine-tune	Tighten path within a street	Global 2-Opt across all streets — destroys groupings	Intra-street 2-Opt only — never crosses street boundaries

Section 4 — Re-optimize on Filter Change

`generateOptimizedRoutes` is a pure, stateless function. It takes a property array as input and returns a route array as output. It holds no internal state between calls and has no side effects on external data structures.

When a sales rep applies a filter (by status, score, street, date, etc.) and taps 'Optimize', the calling code passes the filtered property array to `generateOptimizedRoutes`. The function runs the complete mail-carrier pipeline from scratch on that filtered set and returns a fresh route. The previous route is simply discarded.

- No cache invalidation logic is needed.
- No route state needs to be reset before re-optimizing.
- Filtering and re-optimizing are idempotent — running the same filter twice produces the same route.
- The function is safe to call from any React hook, Redux thunk, or async action without worrying about stale state.

Performance Consideration for the Calling Code

The only consideration for the calling code is performance: for very large property sets (500+), the nearest-neighbor TSP on street centroids runs in $O(S^2)$ time where S is the number of streets. For typical door-knocking sessions of 50–200 properties across 10–30 streets, this is imperceptibly fast (under 5ms). No debouncing or lazy evaluation is required for normal usage, though a debounce on rapid successive filter changes is a reasonable UI-layer optimization.

Table 4 — Re-optimization Behavior by Filter Type

Filter Type	Effect on Input Array	Re-optimization Behavior	Notes
Status filter (e.g., Not Home only)	Reduces property count	Full pipeline re-runs on filtered set	Street groupings recalculated from filtered properties
Score filter (e.g., High priority only)	Reduces property count, may eliminate entire streets	Streets with zero remaining properties are dropped from centroid TSP	Single-property streets handled gracefully
Street filter (e.g., Oak St only)	May reduce to a single street	<code>orderStreetsByNearestNeighbor</code> returns trivially; boustrophedon runs on one street	No performance concern
Date filter (e.g., Not visited since Monday)	Variable reduction	Full pipeline re-runs	Idempotent — same filter always produces same route

No filter (all properties)	Full property set	Full pipeline; typical session < 5ms	Baseline case
----------------------------	-------------------	--------------------------------------	---------------

Section 5 — Edge Cases Handled

Table 5 — Edge Case Handling Summary

Edge Case	Handling Function	Behavior	Safe?
Missing street_name (null/undefined/empty)	normalizeStreetName, groupByStreet	Assigned to __UNKNOWN__ group; appended last in route	Yes— no crash, minimal disruption to main route
Missing house_number (null/non-numeric)	boustrophedonStreet	Treated as even-numbered; assigned to even side, sorted by axis coordinate	Yes— safe default; property remains in route
Single-property street	boustrophedonStreet, orderStreetsByNearestNeighbor	Boustrophedon split skipped; single property returned directly; centroid TSP handles as a point	Yes
Street with only odd numbers	boustrophedonStreet	evenSide.length === 0 detected; odd side sorted by axis in arrival direction; no crossing performed	Yes
Street with only even numbers	boustrophedonStreet	oddSide.length === 0 detected; even side sorted by axis in arrival direction; no crossing performed	Yes
Properties with identical coordinates	boustrophedonStreet, generateOptimizedRoutes	Adjacent in sorted order (axis coordinate identical); zero distance causes no division-by-zero; true duplicates removed by deduplication keyed on house_number street_name zip	Yes

Empty input (null/undefined/[])	mailCarrierOrder, generateOptimizedRoutes	Returns empty array immediately; all intermediate functions handle empty arrays without throwing	Yes
------------------------------------	--	--	-----

Missing street_name

Properties with a null, undefined, or empty street_name are assigned to the __UNKNOWN__ group by normalizeStreetName. This group is always appended at the end of the route, after all properly-named streets have been visited. The rep encounters these properties last, minimizing disruption to the main route.

Missing house_number

In boustrophedonStreet, properties with a missing, null, or non-numeric house_number are treated as even-numbered (assigned to the even side). This is a safe default — it prevents a crash and keeps the property in the route. If an entire street has missing house numbers, all properties fall to the even side and are sorted by axis coordinate in arrival direction.

Single-Property Streets and One-Sided Streets

A street with only one property skips the boustrophedon split entirely and returns that single property. The nearest-neighbor TSP handles single-property streets correctly — they are treated as a centroid point and visited in geographic order. For streets with only odd or only even numbers, boustrophedonStreet detects when one side is empty and returns the populated side sorted by axis in the arrival direction. No boustrophedon crossing is performed — the rep simply walks the one side that exists.

Properties with Identical Coordinates

Duplicate coordinates (e.g., two units in the same building) are handled gracefully. They will be adjacent in the sorted order since their axis coordinate is identical. The distance between them is zero, which does not cause division-by-zero errors in any distance calculation. The deduplication logic in generateOptimizedRoutes (keyed by house_number|street_name|zip) removes true duplicates before routing begins.

Section 6 — Performance Notes

Complexity Comparison

Table 6 — Algorithm Complexity Comparison

Algorithm	Status	Complexity	Variables	Notes
global apply2Opt	REMOVED	$O(N^2 \text{ times } I)$	N = total properties, I = iterations (up to 50)	Operated across all streets simultaneously; destroyed groupings
global applyLinkSwap	REMOVED	$O(N^2 \text{ times } I)$	N = total properties, I = iterations (up to 30)	Same cross-street destructive behavior as apply2Opt
orderStreetsByNearestNeighbor	NEW	$O(S^2)$	S = number of streets	Centroid nearest-neighbor TSP; fast for typical $S = 10\text{--}30$
apply2OptCentroids	NEW	$O(S^2 \text{ times } I)$	S = streets, I = iterations (up to 50)	2-Opt on centroid sequence only; never touches property order
boustrophedonStreet (per street)	NEW	$O(K \log K)$	K = properties on one street	Dominated by sort; extremely fast per street
applyIntraStreet2Opt (per street)	NEW	$O(K^2 \text{ times } I)$	K = properties on one street	300-node safety limit preserved; K rarely exceeds 15 in practice
New total	NEW	$O(S^2 + N)$	S = streets, N = total properties	Dominated by centroid TSP + sort across all streets

Practical Impact for FirstKnock Sessions

The following analysis quantifies the reduction in distance calculations for representative session sizes. Distance calculations are the dominant computational unit in both the old and new algorithms.

Table 7 — Distance Calculation Count by Session Size

Session Size	Algorithm	Distance Calculations	Reduction vs. Old
100 props / 20 streets	Old: global apply2Opt	100^2 times 50 =	—
100 props / 20 streets	New: centroid 2-Opt	$\frac{500,000}{20^2}$ times 50 = 20,000	—
100 props / 20 streets	New: intra-street 2-Opt (all streets)	20 times (5^2 times 50) = 25,000	—
100 props / 20 streets	New: total	~45,000	11times fewer than old
200 props / 30 streets	Old: global apply2Opt	200^2 times 50 =	—
200 props / 30 streets	New: centroid 2-Opt	$\frac{2,000,000}{30^2}$ times 50 = 45,000	—
200 props / 30 streets	New: intra-street 2-Opt (all streets)	30 times (6.7^2 times 50) approx 67,000	—
200 props / 30 streets	New: total	~112,000	18times fewer than old

11times45K vs 500K Fewer calculations (100-prop session)	18times112K vs 2M Fewer calculations (200-prop session)	< 5ms Typical execution time (10–30 streets)	300 Node safety limit (preserved from original)
---	--	---	---

Safety Limits Preserved

The 300-node safety limit on applyIntraStreet2Opt is preserved from the original apply2Opt. In practice, no single street in a door-knocking territory will have 300+ properties, so this limit is never hit. It exists as a guard against pathological data (e.g., a malformed import that assigns all properties to a single street name).

Memory

The new algorithm creates no additional data structures beyond the street group map and centroid array. Memory usage is $O(N + S)$, identical to the original implementation. The street group map holds references to the same property objects already in memory — no property objects are copied or cloned during the pipeline.

Summary: Two compounding bugs — property-level inter-street selection and global 2-Opt destroying street groupings — are fully resolved by replacing the ordering pipeline with a centroid-based nearest-neighbor TSP and boustrophedon intra-street walk. The change is a single line in generateOptimizedRoutes. All edge cases are handled. Performance improves by 11–18times for typical session sizes. No state management changes are required in the calling code.